

PUF+SNN

Keegan Hoyne, Will Wallace

Problem & Research Question

Quest 3 applications depend on constant stream of motion data. An attacker can

- Replay data recorded during an earlier session
- Change the data before it is processed
- Make data look like its coming from a different headset or session
- Create weirdmotion patterns that still pass authentication

Previous research is usually examining device authentication, tracking attacks, and machine learning separately. Our project studies how they can work together to protect and analyze these individual XR sensor windows

Initial scope and success criteria

- Begin with Quest 3 head position and quaternion orientation, then add hand tracking later
- Use PUF derived keys to verify each sensor window before SNN inference
- Use the SNN to identify the motion and flag suspicious behavior / patterns
- Evaluate attack rejection, macro F1, false positive rate, and total latency

“Can a PUF based device authentication method detect tampered or replayed Quest 3 head motion data before SNN inference without sacrificing accuracy or adding excessive latency?”

Threat Model

Protected assets

- Headset and session identity
- Integrity of sensor-window
- Window freshness and ordering
- Correct motion and anomaly decisions
- Traceable authentication and inference records

Trust assumptions

- Enrollment database and verifier are trusted
- Cryptographic implementations are correct
- Initial Quest logger and window builder are trusted

Attacker capabilities

- Replay previously valid windows or authentication messages
- Substitute a window from another session or device
- Modify pose samples, labels, timestamps, IDs, or sequence numbers
- Inject noise, bias, drift, jitter, missing samples, freezes, reordering, or pose jumps
- Attempt stale-key use, PUF modeling, cloning, or noisy response attacks

Initially out of scope

- Rooted or modified Quest firmware
- Compromised authentication server
- Invasive PUF extraction and side-channel attacks
- Unsafe physical tracking interference

Authentication to verify the device identity, session, freshness, and protected-window integrity. SNN anomaly detection is able to evaluate if an authenticated sensor pattern looks suspicious.

Literature & Technical Landscape - XR / SNN

Area	What the research found	How it affects our project	Source
XR motion privacy	Head & hand motion identified over 55,000 users across sessions, which shows how sensitive motion data is	We will treat motion as sensitive and separate users, sessions, and devices during evaluation.	VR Motion Identification - Nair et al. [1]
Traditional ML	Random forest and gradient boosted methods such as LightGBM perform strongly on XR motion data.	We need to compare SNN w/ logistic regression, random forest, and LightGBM.	R Motion Identification - Nair et al. [1]
SNN classification	LIF neurons build up info over time and fire spikes when they reach a threshold. Surrogate gradients make them better to train	We will start w/ a small snnTorch LIF model using direct sensor input, then test delta encoding later	Training SNNs - Eshraghian et al. [5]
SNN usual activity detection	An SNN detected unusual patterns across multiple time series signals and produced a continuous anomaly score.	We will add a normal / suspicious score and test it on altered Quest motion windows.	SNN Anomaly Detection - Bäßler et al. [8]
Tracking attacks	VR tracking can be interrupted or manipulated, causing wrong position and orientation data.	We will simulate frozen data, missing samples, jumps, drift, & noise.	VR Tracking Attacks - Rafique and Cheung [6]
Attack investigation	REALITYCHECK tested 25 XR attacks and used structured records to trace what happened during each attack.	We will record the attack type, authentication result, anomaly score, motion prediction, and latency.	REALITYCHECK - Shoaib et al. [7]

Literature & Technical Landscape - PUF / Auth

Area	What the research found	How it affects our project	Source
XR authentication	A device equipped with a PUF, can authenticate with a trusted server and establish a session key. Fuzzy extraction can help handle noisy PUF response.	The PUF will be used to verify which device the sensor data came from. Once the device has been authenticated, a session key is derived. This key is then used to protect the data.	PUF-Based XR Authentication - Cha et al. [2]
Replay protection	Session identifiers, timestamps, expiration values, challenges, and nonces all help with detecting/rejecting replayed and duplicated data before it is trusted.	To bind the sensor windows to its correct device, session, sequence number, timestamps, and payload.	PUF-Based XR Authentication - Cha et al. [2]
PUF modeling attacks	Logistic regression along with other machine learning methods are able to imitate certain PUFs with enough exposed challenge-response pairs.	Protect CRPs and test if a learned model could impersonate the simulated PUF.	PUF Modeling Attacks - Rührmair et al. [3]
PUF reliability	PUF responses are noisy and may not be the same every time. They require secure enrollment, helper data, and error correction to make sure they are unique and identifiable.	Ensure that reliability, uniqueness, bit-error rate, false acceptance, and false rejection are measured correctly.	PUF Tutorial and Applications - Herder et al. [4]
PUF design choices	That a strong PUF supports authentication, where as a weak PUF is commonly used for key generation.	To choose if the project should uses the PUF for direct authentication, key derivation, or both.	PUF Tutorial and Applications - Herder et al. [4]

Research Gap

Existing research addresses the components separately

- PUF research looks at hardware identification, key generation, modeling attacks, and reliability.
- XR research focuses on motion privacy, tracking manipulation, authentication, and auditing.
- SNN research studies temporal classification, anomaly detection, and robustness.
- SNN research looks at classifying motion patterns, anomaly detection, and robustness.

What is missing from the reviewed literature?

- Authentication of the physical Quest sensor windows prior to the SNN inference.
- Evaluation of other attacks, device substitution, protected-window modification, replay, and semantic sensor attacks.
- SNN motion classification and anomaly detection within the same authentication pipeline.
- Measure the combined security, classification, anomaly-detection, and latency. Essentially, how to measure if the system is useful.

Our contribution

- Build a reproducible software prototype anyone could use.
- Test the system using controlled synthetic or scripted attacks.
- Compare four different system versions under the same conditions.
- Document boundaries between the authentication and anomaly detection.

Through all the literature reviewed thus far, not one single study combines device bound, PUF derived authentication of Quest head motion windows using SNN motion classification and anomaly detection while also evaluating replay rejection, sensor attack detection, full latency, and classification performance.

Existing Tools

Technical tools and platforms

- Quest data collection - Meta Quest 3, Unity, OpenXR, and Meta XR packages
 - We will build the Quest 3 logger and collect head positions, quaternion orientation, timestamps, & tracking status
- Deployment / testing - Android Build Support, ADB / Meta Quest developer utilities
 - Install the app on headsets, run trials, transfer logs
- Data pipeline - Python, pandas, NumPy
 - We need to clean data, create 2s windows, validate the schema, perform participant / session / device splits
- Traditional ML baselines - Logistic Regression, random forest w/ SciKit learn, LightGBM
 - Determine whether the SNN compares to simpler linear and tree based ML classifiers
- SNN development - PyTorch and snnTorch
 - We will build the LIF motion classifier and normal / suspicious anomaly detector
- Authentication prototype - Python cryptography library, mock PUF, fuzzy extractions, HKDF, HMAC, or AEAD
 - Simulate PUF session keys as well as authentication of individual sensor windows.
- Evaluation and visualization - scikit learn metrics, Matplotlib, Seaborn
 - Calculate accuracy, macro F1, FPR, latency, attack rejection, and produce plots.
- Development
 - VS Code, Jupyter, Git, Github

Proposed System

Device Trusting & Sensing → 2s window → authentication gate → SNN inference → decision & audit record

Device Trusting & Sensing

- Quest records head position, orientation, timestamps, and tracking status
- PUF response is stabilized and used to derive session key material.
- Session setup occurs once, sensor data is grouped into 2 second windows.

Authentication gate

- Use a keyed tag to protect the window data and metadata.
- The device, session, sequence, freshness, and integrity is checked by the verifier.
- Rejected windows are then logged and only the accepted windows reach SNN.

SNN inference

- Classifies the head motion - nod, shake, left, right, or still / none
- Makes a continuous anomaly score or normal / suspicious decision (still deciding)
- Doesnt perform cryptographic authentication.

Decision & Audit

- Result of authentication and reasoning for rejection.
- Motion prediction with the anomaly score.
- The condition of attack and latency for each of the stages.

The PUF establishes device session trust, each sensor window is authenticated before inference, and the SNN evaluates only accepted motion data.

Quest Sensing & Data Plan

Classifications	Sampling	Model Window	Trial Procedure	Planned Session
nod, shake, look left & return, look right & return, still / no motion	We are targeting 60 Hz while while saving the actual time of each sample	2s action window, 120 samples total (could change)	1s prompt → 2s action → 1 second rest → repeat	15 trials for each motion in random order, after we get approval

Sensor / Experiment data	Head position (x, y, z)	Head orientation(x, y, z, w)	Timestamp and tracking validity from Quest	Participant, session, trial, window ID	Trial stage and correct motion label
--------------------------	-------------------------	------------------------------	--	--	--------------------------------------

Authentication Data	Coded device and session IDs	Window ID / increasing sequence number	Window start / end & data format version	None or key version / authentication tag	Accept / reject result, reason, and latency
---------------------	------------------------------	--	--	--	---

We avoid data leakage by making sure entire trials, sessions, participants, and devices stay in either training or testing. Overlapping windows never cross between the two

Before we get approval we will only use synthetic data or scripted headset runs

PUF Interface

PUF Enrollment

- 3 fixed 255-bit challenge blocks (765 bits in total) , stored within $R_{\text{enroll}, i}$

For each of the block

- Generate a random 63-bit secret M_i
 - Independently random, not derived from the PUF reading.
- BCH(255, 63, t=30): encode M_i into a 255-bit codeword C_i .
- Generate helper data: $W_i = R_{\text{enroll}, i} \oplus C_i$.
- Compute a check value: Hash(M_i)
 - Hash used later to verify a reconstructed M_i is correct; however, it is not used to produce M_i .

Surviving data:

- W_1, W_2, W_3 (helper data), Hash(M_1), Hash(M_2), Hash(M_3)
 - o All on authentication and device layer.
- M_1, M_2 , and M_3 are also stored on authentication layer.

PUF Simulation

Four independent noise sources are modeled

- Manufacturing variation (this is fixed per device and allows for the uniqueness)
- Environmental drift (The shifts in voltage and temperature within a session)
- Long-term drift (aging and shifts across a device's life)
- Measurement noise (random, resets every reading).

Authentication Interface

Session Authentication

- 128 ROs, covers 765 needed challenges.
- 1 challenge = 1 response bit
- 3x independent 255-bit challenges (765 response bits total), fixed for the device lifetime.
- Fuzzy extractor: BCH (255,63, t = 30) per block
- 31–41 responses per authentication check, majority voted per bit, result = R_{majority} .
- Device locally decodes using R_{majority} XOR W_i -> BCH decode -> M'_i . Hash(M'_i) and is checked against device's stored Hash(M_i).
- Verify Hash(M'_i) = Hash(M_i)
- HKDF-SHA256 using $M_1 \parallel M_2 \parallel M_3$, splits into K_{enc} / K_{mac} with distinct context strings.
 - Separate keys for data encoding vs. message authentication.
- Each side (Device vs. Authentication Server) derives keys independently, then checks that an AEAD-encrypted confirmation message is decrypted correctly under its own key.
- 1 session fully authenticated per ~ 60-360 seconds. (PUF challenges once per session)

Sensor Window Protection

- Windows parameters: window_data, timestamp, sequence_number, session_id, nonce
 - Window data is encrypted using AES-256-CTR and the K_{enc} key.
 - Each window contains a tag from HMAC, using the K_{mac} key.
 - All parameters are used to generate this tag.
 - Nonce is unique per window and can be counter tracked (can use sequence_number).
- Verify at gate if each window is clean, using the $K_{\text{mac'}}$, K_{enc} keys, as well as data associated with the window (unencrypted).
 - Decrypt with locally calculated keys (done in the AS), and recalculate the window tag using the ciphertext + AAD + locally calculated $K_{\text{mac'}}$.
 - Check the sequence number and timestamp to prevent replay/stale data.
 - If all pass, forward to SNN or else reject and log.

Attacks Plan

Type of Attack	Examples	Evaluate
PUF & Authentication attacks	Noisy PUF response, replayed login, expired key, or PUF modeling attempt	Whether the PUF stays reliable and authentication rejects invalid attempts
Replay or wrong identity	Replay an old window, use a window from another session, or substitute another device	Whether session IDs, timestamps, and sequence numbers catch it
Changing a protected window	Changing the pose, label, ID, timestamp, or sequence number after tagging	Whether tag verification detects the change
Unusual sensor data	Adding noise, drift, jitter, missing samples, freezes, reordered samples, or pose jumps	Whether the SNN flags the window and still classifies the motion correctly
Targeted model attacks	Make small changes intended to force the wrong motion prediction	How often the SNN catches them and how much its accuracy drops

Evaluation Plan

Four conditions

- Traditional classifier only, with no security
- SNN classifier and unusual behavior detector, w/o PUF authentication
- PUF authenticated data with a traditional classifier
- Complete PUF and SNN system, fully integrated

Outputs

- Authentication - accept / reject
- Motion - nod, shake, left, right, still
- Anomaly - either a score or normal / suspicious

Measurements

- Motion classification - accuracy, macro F1, and confusion matrix
- Unusual behavior detection - precision, recall, F1, false positive rate, and AUROC/PR-AUC
- Authentication - attack rejection, FAR, FRR, and successful key reconstruction
- Speed - median and p95 authentication, inference, total latency

Authentication and the SNN handle different issues - a replay may look normal to the model, and unusual but correctly authenticated data may still need to be flagged by the SNN

Proposed Responsibilities

Keegan - XR / SNN / data pipeline

- Build the Quest logger and define the head motion task
- Design the data format, trial procedure, and dataset splits
- Create sensor attacks and the preprocessing pipeline
- Train the ML baselines
- Build the SNN classifier and unusual behavior detection
- Measure accuracy, robustness, and inference time

Will - PUF / authentication

- Simulate the PUF and test its reliability
- Set up device enrollment and key reconstruction
- Build the session-authentication process
- Protect each sensor window and detect replays
- Test PUF modeling attacks
- Measure authentication success and latency

Shared

- Research question and threat model
- Authenticated-window interface
- 4 system setups
- System integration
- Literature landscape
- Presentation, audit records, and final analysis

First Month Plan

Week	Keegan	Other Student	Shared Deliverable
1	Finalize the task, data schema, trial plan, splits, attacks, metrics, and research notes	Choose the PUF approach and outline enrollment, authentication, attacks, and metrics	One page project specification, presentation
2	Build the Unity/OpenXR logger, generate scripted data, and check the data format	Build a configurable PUF simulator and run initial reliability tests	Shared repository and sample synthetic data
3	Prepare the approved or scripted dataset, add attack transformations, and train logistic regression and random forest	Build enrollment, key reconstruction, session authentication, and window tagging	Initial classification, PUF, and authentication results
4	Train snnTorch LIF baseline and conduct first robustness test	Test key reconstruction and connect window verification to the pipeline	First end to end PUF gated inference run

Faculty Questions

- Are the Quest 3 headsets already set up for development through the lab?
- Are we expected to download these applications on the Lab computer or our personal laptops? Or does it not matter?
- Who can give us permission to install and test software on them?
- What work can we do before receiving IRB approval?
- How much added delay would be considered acceptable?
- How should we store and eventually delete the Quest motion data?
- When should we begin validating on FPGA hardware?

References

- [1] V. Nair et al., "Unique Identification of 50,000+ Virtual Reality Users from Head & Hand Motion Data," in Proc. 32nd USENIX Security Symposium (USENIX Security '23), 2023, pp. 895–910. Accessed: Aug. 31, 2026. [Online]. Available: <https://www.usenix.org/system/files/usenixsecurity23-nair-identification.pdf>
- [2] W. Cha, H. J. Lee, S. Kook, K. Kim, and D. Won, "A Lightweight Authentication and Key Distribution Protocol for XR Glasses Using PUF and Cloud-Assisted ECC," *Sensors*, vol. 26, no. 1, Art. no. 217, 2026, doi: 10.3390/s26010217.
- [3] U. Rührmair, F. Sehnke, J. Sölter, G. Dror, S. Devadas, and J. Schmidhuber, "Modeling Attacks on Physical Unclonable Functions." Accessed: Aug. 31, 2026. [Online]. Available: <https://eprint.iacr.org/2010/251.pdf>
- [4] C. Herder, M.-D. Yu, F. Koushanfar, and S. Devadas, "Physical Unclonable Functions and Applications: A Tutorial," *Proceedings of the IEEE*, vol. 102, no. 8, pp. 1126–1141, Aug. 2014, doi: 10.1109/JPROC.2014.2320516.
- [5] J. K. Eshraghian et al., "Training Spiking Neural Networks Using Lessons From Deep Learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, Sep. 2023, doi: 10.1109/JPROC.2023.3308088.
- [6] M. U. Rafique and S. C. S. Cheung, "Tracking Attacks on Virtual Reality Systems," *IEEE Consumer Electronics Magazine*, vol. 9, no. 2, pp. 41–46, Mar. 2020, doi: 10.1109/MCE.2019.2953741.
- [7] M. Shoaib, A. Suh, and W. U. Hassan, "Principled and Automated Approach for Investigating AR/VR Attacks," in Proc. 34th USENIX Security Symposium (USENIX Security '25), 2025, pp. 4325–4344. Accessed: Aug. 31, 2026. [Online]. Available: <https://www.usenix.org/system/files/usenixsecurity25-shoaib.pdf>
- [8] D. Bäßler, T. Kortus, and G. Gühring, "Unsupervised Anomaly Detection in Multivariate Time Series with Online Evolving Spiking Neural Networks," *Machine Learning*, vol. 111, pp. 1377–1408, 2022, doi: 10.1007/s10994-022-06129-4